

Week 8: Cloud Computing

Python 200 I Mentor Session

What This Week Covers

Students have spent seven weeks building things locally — models, pipelines, AI tools — all running on a laptop. This week shifts the context.

- **What cloud computing is** — and why almost all production data work runs there
- **IaaS, PaaS, SaaS** — the spectrum from raw infrastructure to managed services
- **The provider landscape** — hyperscalers, developer platforms, BaaS
- **Why Supabase** — the rationale for our hands-on tool in weeks 9–11

The goal is conceptual fluency: enough to read a job description, evaluate a tool, and understand the context when they join a team that uses cloud infrastructure.

The Core Idea

Instead of buying and maintaining your own servers, you **rent** computing resources from a provider and pay for what you use.

- Need a GPU cluster to train a model? Rent it for an afternoon, then shut it down.
- Need to store a terabyte of data? No hard drives required.
- Need global availability? Pick a region — the provider handles the infrastructure.

This pay-as-you-go model, combined with virtually unlimited scale, is why cloud computing became the default environment for data-intensive software.

Discussion: Before the cloud, a startup that wanted to launch a product had to buy and rack servers. What does that mean for who can build software, and how fast?

IaaS, PaaS, and SaaS

The spectrum is defined by one question: *how much does the provider manage for you?*

IaaS (Infrastructure as a Service) — raw compute, storage, network. You set up everything else: OS, software, security. Most flexible, most work. *Example: AWS EC2, Azure VMs*

PaaS (Platform as a Service) — provider manages the infrastructure; you bring your code. You deploy, they handle running it and scaling it. *Example: AWS Elastic Beanstalk, Azure App Service*

SaaS (Software as a Service) — you open a browser and use an app. Zero infrastructure responsibility. *Example: Gmail, Dropbox, Google Docs*

Scaling: Two Flavors

Scaling is the headline benefit of cloud infrastructure.

Vertical scaling — make one machine bigger: more CPU, more RAM, a larger GPU. Simple, but has a ceiling. Every server has a maximum size.

Horizontal scaling — add more machines and split the work across them. How large services handle traffic spikes: provision additional instances automatically when demand rises, wind them down when it quiets.

You pay for what you use — not for peak capacity sitting idle most of the time.

Discussion: A web app serves 1,000 users on a normal day and 50,000 users during a product launch. Which scaling approach handles that better, and why?

Cloud Limitations

The cloud isn't the right tool for every problem.

- **Cost can spiral** — leaving a GPU cluster running overnight or forgetting to clean up storage generates surprise bills. Always set budget alerts.
- **Local is sometimes faster and cheaper** — if your dataset fits on one machine and you have no massive compute needs, local is often the better prototype environment.
- **The learning curve is steep** — cloud platforms are enormous ecosystems with hundreds of services. Even experienced engineers look things up constantly.
- **AI advice may be stale** — cloud platforms change fast. LLMs are often behind on current service names, pricing, and API behavior.

Patience and comfort with documentation matter as much as Python skills in cloud environments.

The Big Three Hyperscalers

"Hyperscaler" = planetary-scale cloud provider. There are three.

Amazon Web Services (AWS) — oldest and largest (>35% market share). Broadest service catalog. Default choice for most startups and many enterprises.

Google Cloud Platform (GCP) — strongest in data and ML. Invented many foundational distributed systems ideas (MapReduce, BigQuery's predecessor). Preferred for large-scale analytics.

Microsoft Azure — dominant in enterprise and government. Deep integration with Windows, Active Directory, and Microsoft 365. Many large nonprofits already have Azure agreements.

All three offer roughly equivalent capabilities. The main driver for picking one is usually: *what does your organization already use?*

Beyond the Hyperscalers

Developer-tier platforms — fewer services, much simpler setup.

- **DigitalOcean** — simple, predictable pricing (\$6/month VMs). Popular with small teams.
- **Render / Fly.io** — take a Git repo or Dockerfile and deploy it automatically. Managed Postgres is first-class.
- **Cloudflare** — CDN + edge compute + S3-compatible storage (R2) with no egress fees.

Backend-as-a-Service (BaaS) — application-level primitives pre-wired and accessible via API.

- **Firebase** (Google) — NoSQL document store, auth, hosting. Popular for mobile and frontend-heavy apps.
- **Supabase** — open-source Firebase alternative built on **PostgreSQL**. Real relational database plus auto-generated REST API, auth, and file storage.

Why Supabase for This Course

This course originally used Microsoft Azure. We moved to Supabase for three reasons.

Access — Azure requires organizational provisioning: joining a tenant, waiting for an invitation. Supabase accounts are self-provisioned at supabase.com in under two minutes. No waiting, no blocked students.

Pedagogical fit — Azure Blob Storage stores opaque files organized by path. Supabase stores rows and columns in a relational database. Querying, filtering, and reasoning about structured data is a skill used in almost every data role.

Pipeline coherence — the ETL pipeline in weeks 9–11 has a raw zone and an enriched zone. In Supabase, these are two tables with a clear relationship. The structure makes pipeline stages easy to inspect and debug.

Discussion: A Supabase table and an S3 bucket can both store weather data. What can you do with the table that you can't do with the file?

Lesson 1 Demo: Cloud Overview

Open: Lesson 1 → cloud overview reading and videos

Focus on	Skim for now
IaaS / PaaS / SaaS distinction — the key question is "what do I manage?"	Exact service names across providers
Vertical vs horizontal scaling — when each applies	Pricing specifics
Cloud limitations — cost, learning curve, stale AI advice	Azure-specific material in lesson (replaced by Supabase)

Have students identify where a tool they've used (GitHub, Colab, OpenAI API) falls on the IaaS/PaaS/SaaS spectrum.

Discussion: The lesson says cloud platforms are "notorious for how fast they change." What does that mean for how you should use AI tools when debugging cloud setup issues?

The Service Taxonomy

When you encounter a new cloud provider, look for these categories:

Category	What it does	AWS	GCP	Azure	Supabase
Compute	Run code on a server	EC2	Compute Engine	VMs	—
Serverless	Functions, no server management	Lambda	Cloud Functions	Azure Functions	Edge Functions
Object storage	Store files by key	S3	Cloud Storage	Blob Storage	Storage
Managed relational DB	Hosted SQL database	RDS	Cloud SQL	Azure SQL	✓ (core product)
LLM API	Access to large language models	Bedrock	Vertex AI	Azure OpenAI	—
ML platform	Train and deploy models	SageMaker	Vertex AI	Azure ML	—

Most projects don't use one provider for everything.

Lesson 2 Demo: Provider Landscape

Open: Lesson 2 → cloud landscape reading

Focus on	Skim for now
Why Supabase for this course — all three reasons	Firebase vs Supabase internals
The service taxonomy table — recognizing categories, not memorizing names	Specific pricing or free tier limits
Hyperscaler vs developer platform vs BaaS — where each sits	Cloudflare Workers and edge compute

Ask: can anyone name a service they've already used in this course that's "cloud"? (OpenAI API, GitHub — both are SaaS/PaaS.)

Discussion: A classmate says "I'll just use Azure because it's what big companies use." What would you say? What factors actually drive the choice?

Week 8 Wrap-Up

- **IaaS / PaaS / SaaS** — the spectrum from raw infrastructure to fully managed apps; the key question is "what do I manage?"
- **Scaling** — vertical (bigger machine) vs horizontal (more machines); cloud handles both on demand
- **Cloud economics** — pay for what you use; cost spirals are real; local prototyping is often the right starting point
- **Hyperscalers** — AWS (broadest), GCP (data/ML), Azure (enterprise/government)
- **Supabase** — hosted Postgres, self-provisioned, relational structure maps naturally to the ETL pipeline ahead
- **Service taxonomy** — compute, serverless, object storage, managed DB, LLM API, ML platform

Takeaway: The cloud is not one thing — it's a spectrum of services with different tradeoffs. Knowing the categories is more important than memorizing specific service names. Starting in Week 9, you'll use Supabase as the cloud data layer for a real ETL pipeline.